

## **Laboratorio #2: Representaciones básicas de texto**

### **1. Bolsa de palabras (Bag of Words)**

Se construyó la representación de Bolsa de Palabras del corpus normalizado usando CountVectorizer de scikit-learn. La matriz resultante tiene una forma (shape) de 1138 x 23409, es decir 1138 documentos y un vocabulario de 23409 palabras distintas.

Con esta representación se pierde por completo el orden de las palabras, la gramática y el contexto en el que aparece cada una. Por ejemplo, si a una oración se le quita una palabra clave como un “no”, Bag of Words no distingue negaciones ni la posición de las palabras dentro de la oración, entonces el significado se puede perder o hasta invertir sin que la representación lo note.

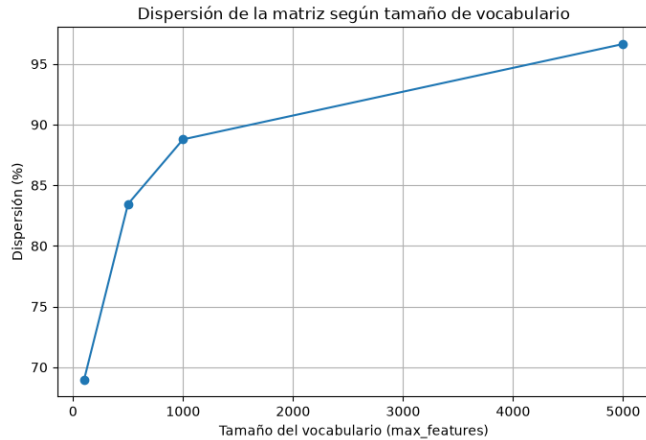
Lo que se gana a cambio es que se vuelve una representación mucho más eficiente: es rápida de calcular, fácil de entender, y funciona bien para tareas como clasificación de texto, porque muchas veces con solo saber qué palabras aparecen en un documento, sin importar el orden, ya alcanza para distinguir de qué tema habla.

### **2. Vectores de conteo y dispersión**

A partir de la matriz de conteos anterior se calculó qué porcentaje de las celdas son ceros. El resultado fue una dispersión (sparsity) de 99.17%, es decir que prácticamente toda la matriz está llena de ceros y solo una fracción mínima de las celdas tiene un valor distinto de cero.

También se graficó cómo cambia la dispersión al limitar el vocabulario a distintos tamaños: con 100 palabras la dispersión es de 68.96%, con 500 palabras sube a 83.46%, con 1000 palabras llega a 88.78%, y con 5000 palabras alcanza 96.64%. Se ve claramente que entre más grande el vocabulario, más dispersa se vuelve la matriz.

Esto pasa porque cada documento sigue teniendo más o menos la misma cantidad de palabras propias, pero el espacio de columnas disponibles sí crece. Si un documento usa 150 palabras distintas y el vocabulario total es de 500, ese documento llena una parte considerable de las columnas; pero si el vocabulario crece a 23409, ese mismo documento ocupa una porción mínima de él y el resto queda en cero. Esto tiene implicaciones directas en almacenamiento y cómputo: guardar todos esos ceros desperdicia muchísima memoria, y hacer cálculos sobre una matriz gigante llena de ceros es mucho más exigente para la computadora si no se maneja de forma inteligente.



### 3. N-gramas

Los 15 bigramas más frecuentes del corpus normalizado fueron:

- banco central (219)
- poder ser (201)
- bbva research (200)
- tasa interés (189)
- cada vez (184)
- entidad financiero (160)
- cambio climático (157)
- punto básico (139)
- año pasado (136)
- bbva México (134)
- precio consumidor (134)
- si bien (112)
- último año (111)
- largo plazo (104)
- hacer él (100)

Varios de estos bigramas son términos que, si se separan y se leen como palabras individuales, pierden sentido. Por ejemplo, banco central, tasa interés, cambio climático y precio consumidor son términos económicos que, si se separan en dos palabras sueltas, pierden esa unidad de significado que sí tienen juntos.

Una ventaja de los bigramas es que dan un poco de contexto y orden local, permitiendo distinguir frases con significado propio que como palabras sueltas no significan lo mismo. El costo es que el vocabulario se vuelve demasiado grande: se pasó de 23409 a 227577 términos, casi 10 veces más, lo que también implica que la matriz de bigramas termina siendo mucho más dispersa que la de unigramas.

#### 4. TF-IDF

Se construyó la representación TF-IDF del corpus normalizado usando `TfidfVectorizer`, obteniendo una matriz de forma 1138 x 23409, el mismo tamaño de vocabulario que con la Bolsa de Palabras.

Se seleccionaron 3 documentos de categorías distintas y se revisaron sus 10 palabras con mayor puntaje TF-IDF. En el documento de categoría Otra destacaron negocio, garcía, sostenibilidad, físico, sostenible, prioridad, imaginamos, compitar, articulador y acá. En el documento de categoría Regulaciones destacaron chino, cotizar, extranjero, sec, csrc, denegar, empresa, regulador, estadounidense y decir. En el documento de categoría Alianzas destacaron viva, aerolínea, aerobus, us50, aviación, low, cost, avión, pasajero y vivo.

Estas palabras son las que mejor distinguen a cada documento del resto del corpus. Por ejemplo, en el documento de Alianzas, el top de TF-IDF trae viva, aerobus, us50, low, cost, términos bien específicos de esa noticia en particular (viva parece ser el nombre de una aerolínea). Esto refleja que el puntaje de TF-IDF no solo mide qué tanto se repite una palabra dentro del documento, sino qué tan específica o distintiva es comparada con el resto del corpus.

#### 5. Similitud entre documentos (similitud coseno)

Usando los vectores TF-IDF se calculó la similitud coseno entre pares de documentos con `cosine_similarity` de `scikit-learn`. Se tomó como referencia el documento índice 0, de categoría Otra, y se buscaron los 5 documentos más similares dentro de todo el corpus. Los resultados fueron: documento 545 (Otra, similitud 0.3181), documento 1027 (Sostenibilidad, similitud 0.2384), documento 1008 (Otra, similitud 0.2202), documento 547 (Sostenibilidad, similitud 0.2191) y documento 859 (Sostenibilidad, similitud 0.2150).

```
Documento de referencia: índice 0, categoría: Otra

Los 5 documentos más similares:
Índice 545 - Categoría: Otra - Similitud: 0.3181
Índice 1027 - Categoría: Sostenibilidad - Similitud: 0.2384
Índice 1008 - Categoría: Otra - Similitud: 0.2202
Índice 547 - Categoría: Sostenibilidad - Similitud: 0.2191
Índice 859 - Categoría: Sostenibilidad - Similitud: 0.2150
```

43] ✓ 6.1s

```
· Similitud promedio entre documentos de la MISMA categoría: 0.0749
  Similitud promedio entre documentos de DISTINTA categoría: 0.0401
```

#### 6. Análisis

¿Cómo cambia la dispersión de la matriz al pasar de unigramas a bigramas o trigramas? ¿Por qué?

La dispersión sube bastante. Se vio que el vocabulario de bigramas (227577 términos) es casi 10 veces más grande que el de unigramas (23409 términos), y cada documento sigue usando solo una fracción pequeña de todas las combinaciones posibles de 2 palabras. Entonces entre más grande el vocabulario, más ceros hay en la matriz, y con trigramas este efecto sería todavía más fuerte.

**De las tres representaciones trabajadas (BoW, n-gramas, TF-IDF), ¿cuál le parece más adecuada para una tarea de clasificación de noticias por categoría? Justifique.**

TF-IDF me parece la más adecuada para clasificar noticias por categoría. La razón es que resalta las palabras que realmente distinguen a cada documento (como vimos con viva y aerobus en la noticia de la aerolínea, o chino y cotizar en la de regulaciones), y al mismo tiempo le baja el peso a palabras genéricas que se repiten en todo el corpus, como poder o año, que no ayudan a diferenciar entre categorías. Bag of Words simple no hace esta distinción, entonces palabras genéricas terminan pesando igual que palabras específicas del tema, y los bigramas por su lado aportan algo de contexto extra.

**¿La similitud coseno agrupa razonablemente los documentos por categoría? ¿Qué categorías se confunden más entre sí y por qué cree que ocurre?**

Sí agrupa de forma razonable, aunque no perfecta. La similitud promedio entre documentos de la misma categoría (0.0749) es casi el doble que entre categorías distintas (0.0401), entonces en general el TF-IDF sí está capturando algo del tema de cada documento. Sin embargo, en el ejemplo del documento de referencia (categoría Otra), 3 de los 5 documentos más similares fueron de la categoría Sostenibilidad y no de Otra. Esto tiene sentido porque Otra es una categoría bastante general que agrupa noticias variadas, y varias de esas noticias terminan hablando de temas relacionados con sostenibilidad.

**Si tuviera que elegir una sola representación para construir un buscador de noticias similares dentro de este corpus, ¿cuál usaría y por qué?**

Usaría TF-IDF combinado con similitud coseno. Ya vimos que TF-IDF resalta bien las palabras distintivas de cada noticia en vez de solo contar repeticiones, y la similitud coseno permite comparar documentos de cualquier longitud de forma justa, sin que una noticia más larga parezca automáticamente menos parecida solo por tener más palabras. Es una combinación clásica y bastante sólida para este tipo de tarea, y con los resultados que se obtuvieron (mayor similitud dentro de la misma categoría que entre categorías distintas) queda claro que funciona razonablemente bien sobre este corpus.

LINK PARA EL GITHUB: <https://github.com/luispegr25/NLP>